

DTSC 691

Machine Learning

Project Submission

Erik Wagner

Introduction

Baseball has recently become an analytical goldmine, as showcased in the 2011 film Moneyball, where a team with limited money uses data analytics to identify undervalued players. The true creator of this era of analytical baseball is a man named Bill James, who authored more than 2 dozen books on the statistics of baseball, coining a term known as sabermetrics. His influence in baseball became so widespread that in 2006, Time Magazine named him as one of the 100 most influential people in the world, nearly 30 years after his first published book on the topic.

Now, every team in professional baseball has entire divisions dedicated to analytics, who are all trying to identify the next trend that can give their team an advantage. This use of data science has become such a trend in fact that major league baseball actually has had to ban some of the strategies that data science has provided.

Goals of the project

With this being the true Golden Age of data science and machine learning in baseball, there is an opportunity to be a part of this in an oft overlooked side of the game. One of the most common machine learning projects that can be seen regarding baseball is a pitch predictor, but rarely, rarely is the other side of the same situation ever seen.

The aim of this project will be to answer the question “What is the optimal strategy for a baseball pitcher in any given situation?” As a secondary goal, clustering will be used to classify pitch type, which will be vital in achieving an optimal strategy.

The aim of the project can more simply be seen as finding the pitch type, horizontal location, and vertical location that will minimize a batter’s opportunity for a successful at-bat in any given situation.

Further Insight and Motivation

Luckily, Major League Baseball has set up 12 ultra high-speed cameras at each baseball stadium, which are used to create a massive amount of data that the MLB releases every night, commonly referred to as Statcast. This data includes common baseball statistics that are tracked, along with much more detailed pitch-by-pitch analysis, which is what this project will primarily be utilizing. Through various machine learning techniques, including clustering and regression this project will be able to identify a given pitcher's arsenal (what types of pitches he throws), identify tendencies of both hitters and pitchers in various states, and ultimately use those to identify the best strategies for a pitcher.

The idea of an optimal pitch will depend on many factors. But, ultimately, the optimal strategies for a pitcher will be decided by two main factors; the pitch type and the location of the pitch. These factors are important independently, but generally will have to work jointly together.

This optimal strategy will change every pitch in an at-bat based on the count, which is the number of balls or strikes thrown to a batter. This strategy can also be affected by many other factors, such as which hand a pitcher throws with, which side the batter stands on, and whether there are runners on base. All of these factors will have to be considered while completing the project.

My major motivation for this project comes from the fact that this has been attempted by others, but in such a way that it would never yield accurate results. Some of the major factors that I changed was adding a machine learning clustering algorithm to identify pitches, clustering hitters and pitchers based on tendencies, and treating this as a regression problem. One such of these projects ignored major influences, such as a pitcher throwing with his left hand vs a pitcher throwing with his right hand. This is something I made sure to include in the project, as I felt like it is imperative. Something seemingly as simple as this actually changes everything, from where a pitcher stands, where he releases the ball, to the direction pitches move, to what pitches a batter will prefer. I feel as though my key changes to implementation are focused on supervising models more closely because without a model understanding baseball, all situations will be treated as equals, which they are not.

Data description

For this project, I will use pybaseball, which is an open-source package that has scraped data from three of the largest baseball data websites, FanGraphs, Baseball Reference, and Baseball Savant. This package will contain data on every pitch for every game since 2017, which is what I will be utilizing as my data source.

Source Name	Source Link
Pybaseball	https://github.com/jldbc/pybaseball
Baseball Savant	https://baseballsavant.mlb.com/statcast_search
FanGraphs	https://www.fangraphs.com/
Baseball Reference	https://www.baseball-reference.com/

The data gathered is both on a broad scale of major statistics for each player, down to exact information on each specific pitch thrown. While all data has to be considered, the primary data statistics will be include:

Object Name	Definition
pitch_type	The type of pitch as named by Statcast
release_speed	pitch velocity
pfx_x	Horizontal movement of pitch in feet
pfx_z	Vertical movement of pitch in feet
spin_axis	The Spin Axis in the 2D X-Z plane in degrees from 0 to 360
effective_speed	Derived speed based on the extension of the pitcher's release.
zone	Location of the ball when it crosses home plate

p_throws	Which hand does the pitcher throw with
stand	Which side the batter stands on
balls	Pre-pitch number of balls in the count
strikes	Pre-pitch number of strikes in the count
on_3b	Is a runner on 3rd base
on_2b	Is a runner on 2nd base
on_1b	Is a runner on 1st base
inning	Pre-pitch inning number.
release_spin	Spin rate of pitch
outs_when_up	Pre-pitch number of outs.
at_bat_number	Plate appearance number of the game.
pitch_number	Total pitch number of the plate appearance
delta_run_exp	The change in Run Expectancy before the Pitch and after the Pitch

There are many other statistics that have the possibility of being utilized, but the ones listed above are my primary concern. These statistics will give me pitching stats such as the release point of each pitch, how fast it is spinning, and how much the pitch moved, as well as hitter stats that will give me an estimate on how likely the hitter was to succeed on a given pitch.

Software

To complete this project, I primarily worked with Python in Jupyter Notebooks as a testing environment, but then used Visual Studio Code as the IDE to deploy a final project version to Flask. This allowed me to make use of Pybaseball, which is an excellent resource for this task. Python has all the capabilities to analyze and ultimately

visualize the data that I need. I then deployed this to Flask and utilized PythonAnywhere to deploy the web app.

Analyses & Model Specifications

To try to achieve the goals, a python package named Pybaseball will be used to obtain necessary data from a few sources. From there, data will be cleaned, then standardized, since common baseball statistics come in such widely varying values. To be able to achieve the best predictions for strategy, first pitch movement data must be tracked and clustered to create organic pitch types because it has been estimated that 10% of all pitches are mislabeled by Statcast. For this, I will try a few clustering algorithms, including K-means, DBSCAN, and Gaussian Mixture Model.

To achieve the goal, I used a python package named Pybaseball to obtain all of the data I needed. This came in the form of 3 different called data sets, one which would be the most effective for clustering pitches together, one which gives general pitch data about a pitcher, and a third which does the same for the batter. First, I was able to estimate the number of unique pitches a pitcher may throw using the elbow method, which compares a sum of squares to values of K. This allowed me to look for an 'elbow', which is the location on the graph where the line begins running parallel to the x-axis. From there, I tested K-means, TSNE, Agglomerative, Gaussian Mixture, and DBSCAN clustering.

While I was successful in my ability to cluster pitches and ultimately identify new clusters of pitches, the data source became a limiting factor as their function for collecting the data needed began raising errors on their end. While this was a minor setback for my algorithm, it in no way diminished the workload of the project as I am ready to include that when the data can be received consistently.

While ideally any predictions would be made by using data of a specific batter against a specific pitcher, realistically, there won't be enough data for all possible situations. Because of the lack of data in very specific situations, creating broader groups classifying types of hitters and types of pitchers will be ideal. For this I chose to focus on concrete classifications for batters and pitchers, which is splitting them by what hand they throw with, or which side of the plate they stand on while hitting. While there were hopes of splitting each group down further, in many cases, it would not leave enough data to provide an accurate model.

After classifying pitches and identifying groups of hitters and/or pitchers, there are two considerations on how to consider ideal pitch strategy. One of these methods, which follows more standard baseball logic, would consider a player's at-bat against a pitcher

as a sequence of pitches, with the previous pitch altering the strategy for the following pitch. While this is conventionally accepted in baseball, it is possible that each pitch should be treated alone, so this thought will be considered. I was able to do this through the use of a new previous pitch column, which would help use part of the sequence a batter had faced to help predict the ideal pitch strategy. Surprisingly, I did not find that the addition had much effect on the results, but I ultimately decided to keep this in as it can be considered a major factor because the last pitch will always be on the mind of a pitcher or hitter.

The final model in answering the question of the most ideal strategy for baseball pitchers is a regression model. This is not the model I expected to be best for the task, but as I looked at the best way to weigh the failure of a batter and success of a pitcher, I quickly found that the results were going to be continuous. The idea of classifying a best pitch or worst pitch became illogical as each of those can vary so widely depending on the batter, pitcher, and overall situation.

Originally, this was expected to create two visualizations, but instead, the two visualizations have been combined into one heat map focused on showing the potential pitch zones and pitch type that will be most effective for the pitcher. The results are very much a combination of both visualizations below.

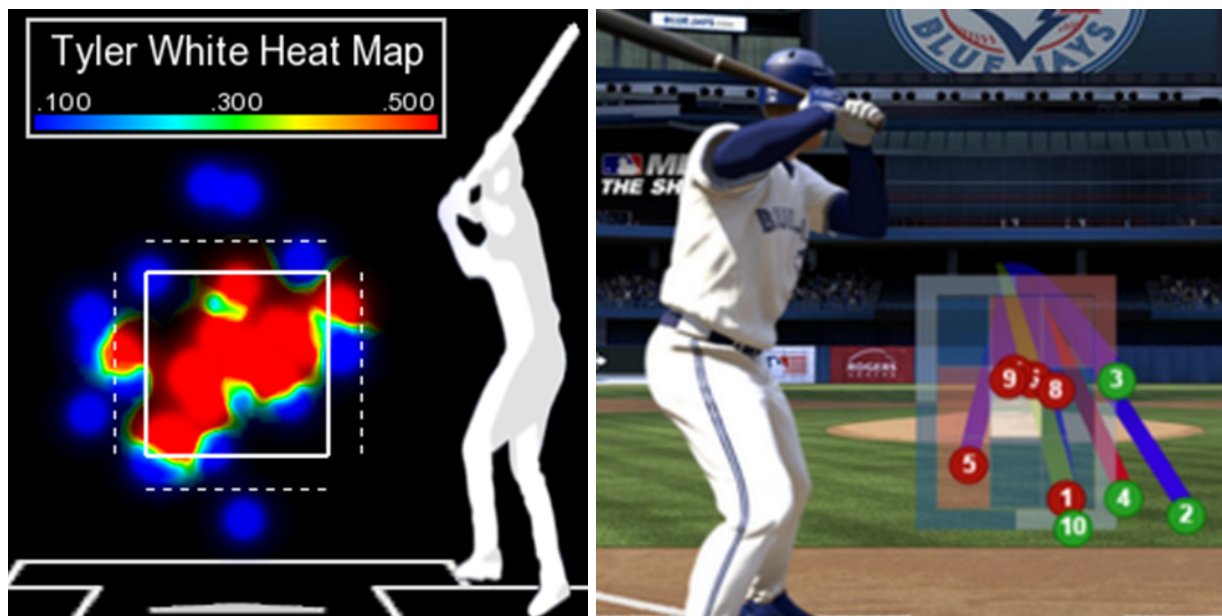


Figure 1: Examples of a common heat map and strike zone visualization

All of the testing for the project will be done in python using Jupyter Notebooks. Due to the somewhat gray nature of results in baseball, I decided to use a 80-20 split for my training and test data to ensure as much training can be done as possible. This will still maintain a great deal of data for training, but will also allow enough test data to help account for the nature of pitching, which is often times centered around being unpredictable. After the testing phase, I was able to move to VSCode and work further on putting together the whole algorithm and web app using flask, which was able to be deployed at ekwag.pythonanywhere.com.

Deliverables

For this project, the most ideal way to present the topic is through the use of my models being deployed via Flask and then also having a walkthrough of the accompanying Jupyter Notebooks. I feel as though this is the correct method of communication because using Flask to deploy the models will allow anyone to see it in action. A baseball situation will be able to be generated automatically or manually entered, and run, which should create a visual that is predicting the most optimal location and pitch type using the given batter.

I feel as though the walkthrough is an imperative method of communication because baseball nomenclature can be very dense with many acronyms. A video walkthrough will allow a quicker and clearer path through what has been accomplished, without the hangups of trying to identify every acronym, remember what that means, and how it has affected the situation.

Ultimately, I decided to deliver the web framework, jupyter notebooks file with more detail and text talking about what I did, and 2 videos; a complete walkthrough and a walkthrough of more specific elements and results.

References

Otremba, S. E. (2022, May 27). *SmartPitch: Applied Machine Learning for professional baseball pitching strategy*. SmartPitch: Applied Machine Learning for Professional Baseball Pitching Strategy. <https://dspace.mit.edu/handle/1721.1/145144>

Dyourri, Abdelhadi. "How to Use Templates in a Flask Application." *DigitalOcean*, DigitalOcean, 13 Sept. 2021, www.digitalocean.com/community/tutorials/how-to-use-templates-in-a-flask-application.

List of Options for Web Application Testing

Pitcher Names-

Clayton Kershaw, Lance Lynn, Jesse Chavez, Adam Wainwright, Rich Hill, Charlie Morton

Batter Names-

Mike Trout, Cody Bellinger, Miguel Cabrera, Joey Votto, Mookie Betts

Inning- 1-9

Outs- 0,1, 2

Balls- 0,1,2,3

Strikes- 0,1,2

On_#b - 0 or 1